

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – SHORT ANSWER TEST

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 3 – Short Answer Test
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	N. Adi Narayana	Reg. No.:	192525346
Section / Batch:	AIML	Date:	02/09/2026

Code of Conduct

I, N. Adi Narayana (192525346) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Adi

Instructions

- This test contains 10 short answer test questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A – Short Answer Test

- Perform the following operations in the Singly Linked List for the following input.
Input: 22-> 77-> 55-> 88->
 - Insert a new-node 110 at the end of the list.
 - Insert a new-node 101 at the position 4 of the list.
 - Delete a node at the beginning of the single linked list
 - Delete a node at the position 3 of the single linked list.
- Rotations in each step of insertion. 1,2,3,4,5,6,7,8,9,10 (), 4) Push (90), 5) Push (36), 6) Push (11), 7) Push (88), 8) Pop (), 9) Pop (). Draw the diagram of the stack and illustrate the above operations and identify where the top is?
- Convert the following infix into a postfix expression. $\rightarrow B - (C/D + (E \% F * G) / H) * J$
- Evaluate the given postfix expression. $53+62/*35*+$
- Construct a B-Tree of Order 3 by inserting numbers 34,26,45,12,87,56,78,22,23,24,65,85,95.
- Construct a 2-3-4 Tree by inserting the following sequence of numbers 45, 36, 76, 23, 89, 115, 98, 39, 41, 56, 69, 48.
- Build a max heap for the following numbers - 20, 35, 23, 22, 4, 45, 21, 5, 42 and 19. At each stage of heap tree construction, ensure heap properties are satisfied.
- Perform the operations onto splay tree (draw after each operation): insert (3), insert (9), insert (12), insert (55), insert (1), insert (3), delete (3), delete (9), delete (60), insert (3), search (12).
- Describe the algorithm to sort the elements 170, 45, 75, 90, 802, 24, 2, 66 using Radix sort. Explain with each phase?
- Consider the following graph & construct BFS.



Graph with six nodes

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	30	Demonstrates complete and accurate understanding of concepts	Good understanding with minor gaps	Basic understanding	Limited understanding
Accuracy of Answer	25	Answers are completely correct and relevant	Mostly correct with minor errors	Partially correct	Mostly incorrect
Problem-Solving & Reasoning	20	Provides logical reasoning and appropriate steps	Good reasoning with minor gaps	Basic reasoning	Lacks logical reasoning
Clarity & Relevance	15	Answers are precise, clear, concise, and directly address the question	Mostly clear and relevant	Somewhat unclear or incomplete	Irrelevant or unclear answers
Time Management & Completion	10	Completes all questions accurately within the time	Completes most questions on time	Requires additional time	Unable to complete the test

SFaculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Singly Linked List Operations

Initial List:

22 $\rightarrow$ 77 $\rightarrow$ 55 $\rightarrow$ 88 $\rightarrow$ NULL

insert a new-node 110 at the end

* Traverse to the last node (88)

* Point its next pointer to the new node (110)

22 $\rightarrow$ 77 $\rightarrow$ 55 $\rightarrow$ 88 $\rightarrow$ 110 $\rightarrow$ NULL

$\Rightarrow$ Insert a new-node 101 at the position 4

* Insert 101 after position 3

22 $\rightarrow$ 77 $\rightarrow$ 55 $\rightarrow$ 101 $\rightarrow$ 88 $\rightarrow$ 110 $\rightarrow$ NULL

$\Rightarrow$ Delete a node at the beginning of singly linked list

* Remove the first node (22)

77 $\rightarrow$ 55 $\rightarrow$ 101 $\rightarrow$ 88 $\rightarrow$ 110 $\rightarrow$ NULL

$\Rightarrow$ Delete node at position 3

$\Rightarrow$ Link the second node (55) directly to the fourth node (88)

77 $\rightarrow$ 55 $\rightarrow$ 88 $\rightarrow$ 110 $\rightarrow$ NULL

Final list $\Rightarrow$ 77 $\rightarrow$ 55 $\rightarrow$ 88 $\rightarrow$ 110 $\rightarrow$ NULL

Algorithm:

* Traverse the linked list

* for insertion, create a new node.

* Adjust the rewired links

* for deletion, change the previous node's link

* Display the final list

(2)

STACK OPERATION.

* Push (30)

* Push (36)

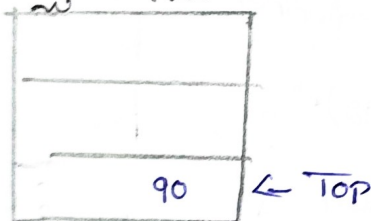
* Push (11)

* Push (88)

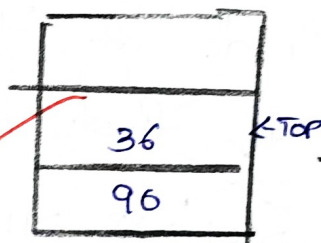
* Pop()

* POP()

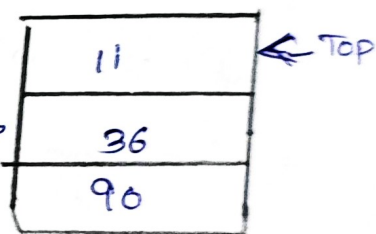
Step 4 push(90)



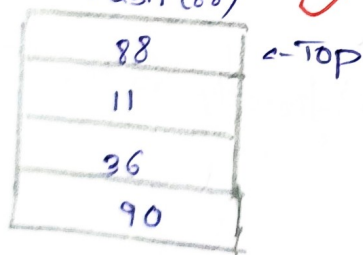
Push (36)



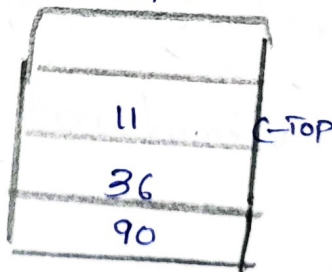
Push (11)



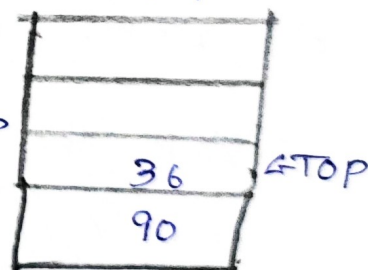
Push (88)



POP()



Pop



finally TOP = 36

Principle LIFO - last in first out

Infix Expression

$$B - (C/D + (E \% F * G) / H) * J$$

Conversion

() * , / , \% → higher priority (left-to-right)
 + - → low priority

Postfix:

$$BCD / EF \% G * H / + J * _$$

you can write it without spaces.

$$BCD / EF \% G * H / + J * _$$

7)

Given postfix $53 + 62 / * 35 * +$
 Use stack.

<u>Token</u>	<u>Action</u>	<u>Stack (Bottom to Top)</u>
5	Push	5
3	Push	5, 3
+	$\text{Pop}(5+3)=8$ Push	8
6	Push	8, 6
2	Push	8, 6, 2
/	$\text{Top}(6/2)=3$ Push	8, 3
*	8×3 Push	24
3	Push	24, 3
5	Push	24, 3, 5
*	$\text{POP}(5 \times 3)$ $24 + 15$ POP	24, 15
+		39

5 B-tree of order 3

Insert : 34, 26, 45, 12, 87, 56, 78, 22, 23, 64, 65, 85, 95

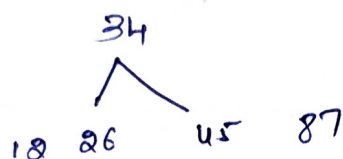
for an order 3. BTree - Maximum children = 3 and maximum

key/s = 2.

After inserting 34, 26, 45

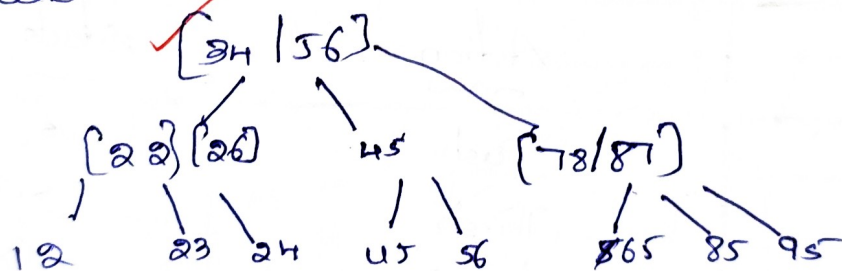


After inserting 12, 87, 56



Continue splitting overflowing nodes.

A valid final structure is



6 2-3-4 Tree

45, 36, 76, 23, 89, 115, 98, 39, 41, 56, 69, 48

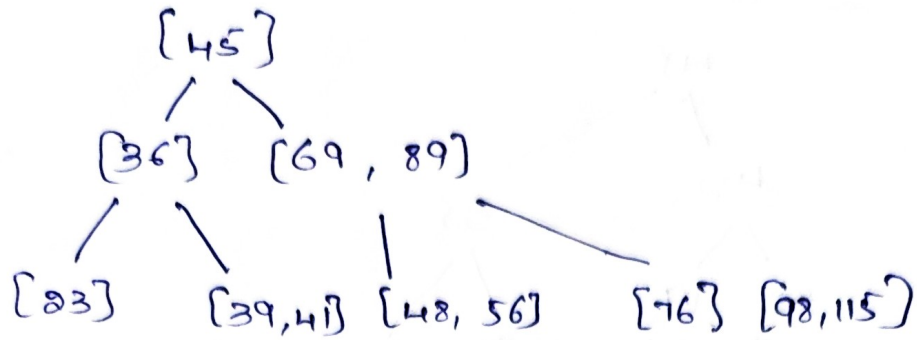
A 2-3-4 Tree can contain

1 key $\rightarrow$ 2-node

2 key $\rightarrow$ 3-node

3 key $\rightarrow$ 4-node.

Tree Structure



7

Max Heap

20, 35, 23, 22, 4, 45, 21, 5, 42, 19

Insert 20

20

Insert 35 $35 > 20$

35

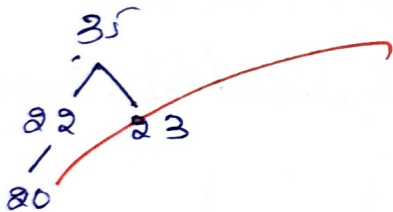
20

Insert 4
 $4 < 20$

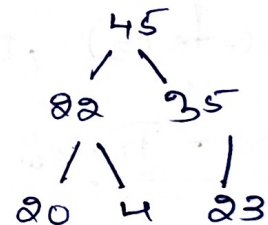
Insert 23
 $23 < 35$

35
20 23

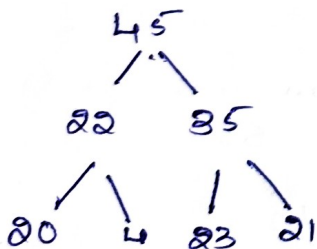
Insert 22 $22 > 20$
 $22 < 35$



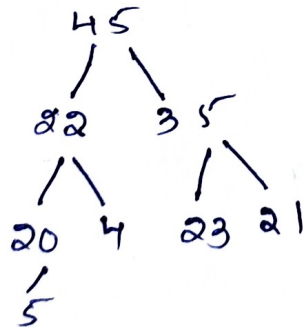
Insert 45
 $45 > 23$ $45 > 35$



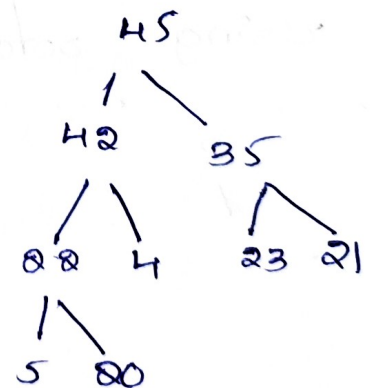
Insert 21 $21 < 35$



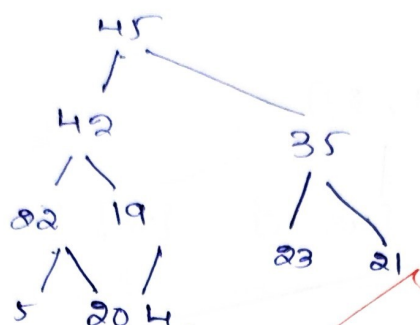
Insert 5



Insert 42



Final Max Heap



Array representation

45, 42, 35, 22, 19, 23, 21, 5, 20, 4

⑧

Splay Tree

Operations

Insert (3), Insert (9), Insert (12), Insert (55), Insert (3),
Delete (3), Delete (9), Delete (60), Insert (3), Search (12)

Import Rule!

In a splay tree, after every insertion, deletion or search the accessed node is moved to the root using rotations.

3
#

9
/
3

55
/
12
/
9
/
3

1
55
12
9
5

for duplicate Insert [33], normally the existing node is
Accessed / spliced rather than creating a duplicate (6)
60 does not exist. ✓

Radix sort

Initial Array : [170, 45, 75, 90, 802, 24, 2, 66]

Phase 1

Extract 1s digit 170, 045, 075, 090, 802, 024, 002, 066

170, 90	802	24	75	66			
0	1	2	3	4	5	6	7

Array: 170, 90, 802, 2, 24, 45, 75, 66

Phase 2

Extract 10s digit: 170, 090, 802, 002, 024, 045, 075, 066

802	24	45	66	75	170	90
0	1	2	3	4	5	6

Array: [802, 2, 24, 45, 66, 170, 75, 90]

Phase 3

Extract 100s digits: 802, 002, 024, 045, 066, 0170, 075, 090

90

75

66

45

24

0

170

1

2

3

4

5

6

7

802

8

9

Final Array: 2, 84, 45, 66, 75, 90, 170, 802

Breadth - first Search

BFS explores the graph level by level starting from a root node

0: connected to 1, 3, 2

1: connected to 0

Queue: [0]

visit 0

⇒ explore neighbours of 0 (1, 2, 3 is added)

Queue [1, 2, 3]

visit 1, 2, 3

⇒ explore neighbours of 1;
neighbours already visited

Queue: [2, 3]

⇒ explore neighbour of 2;
unvisited neighbour is 5

Queue [3, 5]

visit 5

⇒ explore neighbour of 3

unvisited neighbour is 4 add to queue

Queue [4]

visit 4

0 → 1 → 2 → 3 → 5 → 4

0 → 1 → 3 → 2 → 4 → 5 is also a valid.